Azure SQL Database

Interview Questions and Answers for 11+ Years of Experience

Who this is for: senior DBAs, data platform engineers, architects and leads (11+ years) preparing for Azure SQL Database roles. At this level interviewers test **judgment, trade-offs, incident handling and ownership**, not just feature knowledge.

How to use it: Read the answers as talking points, not scripts. For each topic, practice adding one real example from your own experience, with numbers (database size, RPO/RTO achieved, cost saved, incident duration). Parts 1 to 7 cover the technical domains, Part 8 covers design scenarios, Part 9 covers leadership and behavioral questions, and Part 10 is a revision cheat sheet.

Accuracy note: Azure changes quickly. Limits, SKUs and preview status shown here were checked against general Microsoft documentation, but always confirm current numbers on Microsoft Learn before quoting them in an interview or a design review.

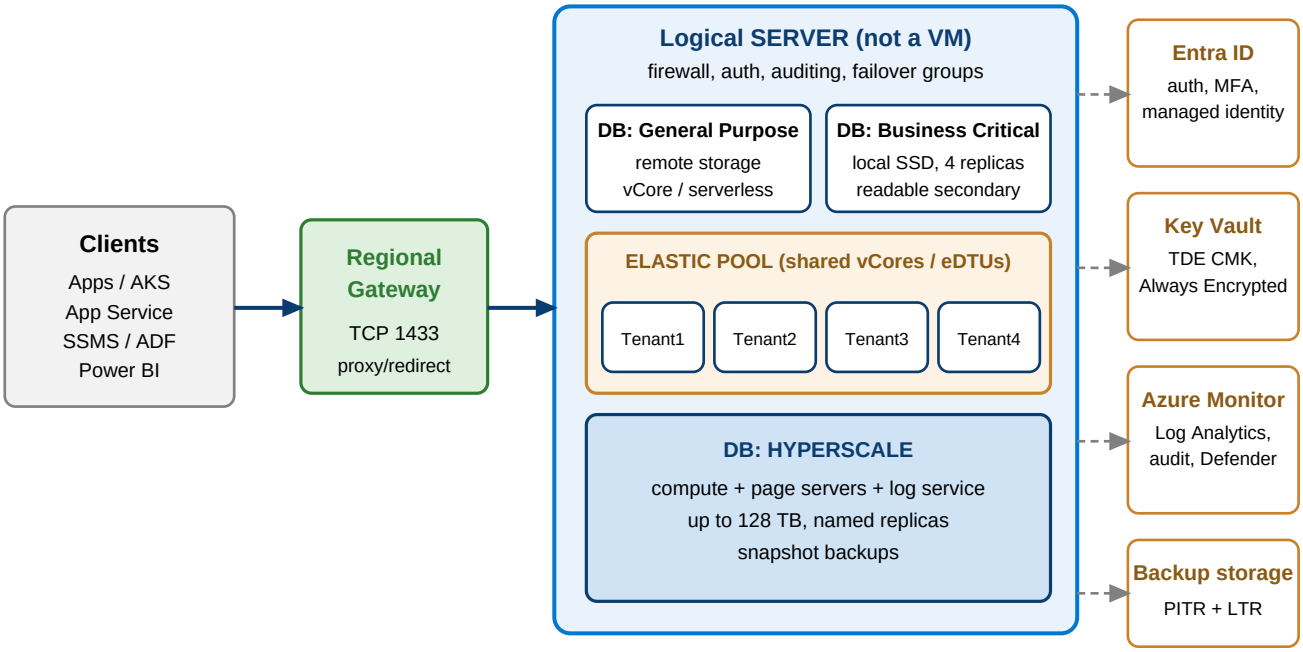
Contents

- Part 1 Architecture and platform fundamentals
- Part 2 Service tiers, compute and scaling (Hyperscale, serverless, pools)
- Part 3 High availability, DR and backups
- Part 4 Security and compliance
- Part 5 Performance tuning and troubleshooting
- Part 6 Migration and modernization
- Part 7 Operations, DevOps, governance and cost
- Part 8 Architecture and scenario-based questions
- Part 9 Leadership and behavioral questions
- Part 10 Cheat sheet (limits, DMVs, error codes, queries)

What interviewers look for at 11+ years	How to show it
Depth plus breadth	Explain internals (why), not just features (what). Connect to real incidents.
Trade-off thinking	Always give options and a recommendation: cost vs resilience, PaaS vs IaaS, Hyperscale vs BC.
Ownership and outcomes	Quote measurable results: RTO achieved, cost cut, tickets reduced, migration size.
Risk and governance mindset	Mention DR drills, security baselines, change control, monitoring and SLOs.
Leadership	Mentoring, standards, influencing app teams and stakeholders, vendor and cost discussions.

Part 1. Architecture and platform fundamentals

Azure SQL Database at a glance



Core architecture

Q1. Explain the Azure SQL Database architecture end to end.

Azure SQL Database is a fully managed PaaS database engine built on the latest SQL Server engine. A client connects through a regional **gateway** (port 1433), which authenticates the request and routes it to the node hosting the database. The database lives on a **logical server**, which is only an administrative and connectivity boundary (firewall rules, logins, Entra admin, auditing, failover groups) and not a VM or an instance. Underneath, each service tier uses a different storage and HA design: General Purpose uses remote storage with stateless compute, Business Critical uses local SSD with an Always On style four-node cluster, and Hyperscale splits compute, log service and page servers. Microsoft owns hardware, OS, engine patching, backups and HA; you own schema, queries, security design, capacity choices and DR design.

Q2. What is a logical server and why does it matter?

It is a management construct that groups databases, elastic pools and failover groups. It holds the server-level firewall, Entra admin, auditing and threat-detection settings, and the connection endpoint `name.database.windows.net`. It matters for design: security policies are applied per server, failover groups are defined per server pair, and region and subscription limits (for example number of servers and DTU or vCore quotas) apply at this level. There is no `master`-level instance access like on-prem; logins for SQL authentication can be created but contained database users are preferred.

Q3. Azure SQL Database vs Managed Instance vs SQL Server on Azure VM: how do you decide?

I use a decision framework. **SQL Database** for cloud-native apps, SaaS and microservices that need elasticity, per-database isolation and the lowest operations overhead. **Managed Instance** when the application relies on instance-scoped features: SQL Agent, cross-database queries and transactions, linked servers, CLR, Service Broker, native .bak restore, or when a lift-and-shift needs minimal code change. **SQL on VM** when you need OS access, third-party agents, SSRS/SSIS/SSAS on the same host, unsupported features or specific versions. Then I validate with an assessment (Azure Migrate or Data Migration tooling), cost model, and operations capacity of the team. The answer is often a mix across the portfolio.

Q4. Describe the connection architecture: gateway, proxy and redirect.

Clients first reach a regional gateway on 1433. In **Proxy** mode all traffic keeps flowing through the gateway. In **Redirect** mode, after the initial handshake the client connects directly to the node, using additional port ranges (11000 to 11999 and 14000 to 16383 depending on setup), which lowers latency and increases throughput. The default is Redirect for connections coming from inside Azure and Proxy for connections from outside. I set the policy explicitly where performance matters and open the required ports for Redirect, and I always use retry logic because node moves change the backend.

Q5. How does Azure SQL Database deliver high availability internally for each tier?

General Purpose: stateless sqlservr compute with data and log files on remote Azure storage; on failure Service Fabric starts compute on another node and reattaches storage, so failover takes a short time. Business Critical: four replicas (one primary and three secondaries) with local SSD using AG-like replication; commits wait for a quorum of secondaries, failover is fastest, and one secondary is readable at no extra cost. Hyperscale: primary compute plus optional HA replicas, backed by the log service and page servers; HA replicas can take over quickly while page servers preserve data. Zone redundancy spreads these replicas across availability zones.

Q6. Which on-prem SQL Server features are missing in Azure SQL Database, and what are the alternatives?

No SQL Server Agent (use Elastic Jobs, Azure Automation, Logic Apps or Data Factory), no cross-database queries except through elastic query or elastic transactions (redesign or use a single database or MI), no CLR or Service Broker, no linked servers, no native BACKUP/RESTORE of .bak files (use BACPAC, DMS or other methods), no FILESTREAM, no SQL Trace or Profiler (use Extended Events), and no instance-level settings or trace flags. I always run an assessment before committing to SQL Database and move blockers into architecture decisions early.

Q7. What is the shared responsibility model for SQL Database and what do you still own?

Microsoft owns datacenter, hardware, host OS, SQL engine patching, automatic backups and built-in HA. I still own data modeling, indexing and query quality, capacity and tier choice, security configuration (identity, network, encryption choices, auditing), DR design and testing, connection resilience in applications, cost control, and operational procedures such as change control and monitoring. A common anti-pattern is assuming PaaS removes the need for performance tuning or DR drills.

Q8. How does patching and maintenance work, and how do you control the impact?

Patching and platform updates are automatic and use rolling reconfigurations, usually producing a brief connection drop for each database. You can set a **maintenance window** (for example overnight in the region time zone) to limit the timing for planned events. Applications must reconnect and retry on transient errors, and long-running transactions can be rolled back during failover. I design for this with retry policies, idempotent operations, short transactions and connection pooling health checks.

Q9. What happens during a reconfiguration or failover and how should applications behave?

Connections are dropped, in-flight transactions roll back, and the database may be briefly unavailable (error 40613 and related transient errors). The application should use a resilient driver setting (`ConnectRetryCount`, `ConnectRetryInterval`), retry with exponential backoff and jitter on known transient errors, avoid caching connections indefinitely, and make writes idempotent. I also test it by forcing a failover in non-prod (Azure CLI or PowerShell) to see the real application behavior.

Q10. How does Azure SQL Database evolve (evergreen model) and what is the impact on versions and compatibility?

The engine is continually updated, so features arrive in the cloud first. Each database has a **compatibility level** that controls behavior; new databases get a recent default but existing ones keep their level until you change it. I treat compatibility level changes as controlled releases: test in a copy with Query Store enabled, compare plans and regressions, then change it during a window with the ability to revert. I avoid depending on undocumented behavior.

Q11. Which defaults differ from on-prem SQL Server that a senior DBA should know?

Read Committed Snapshot Isolation is on by default (row versioning), Query Store is on by default, TDE is on by default with service-managed keys, automatic plan correction is on by default, MAXDOP defaults to 8 for new single databases, backups are automatic and cannot be disabled, auto-create and auto-update stats are on, and many server-level settings are fixed. These defaults are good, but I still validate them, for example RCSI and tempdb or version store usage, and make explicit choices where the workload demands.

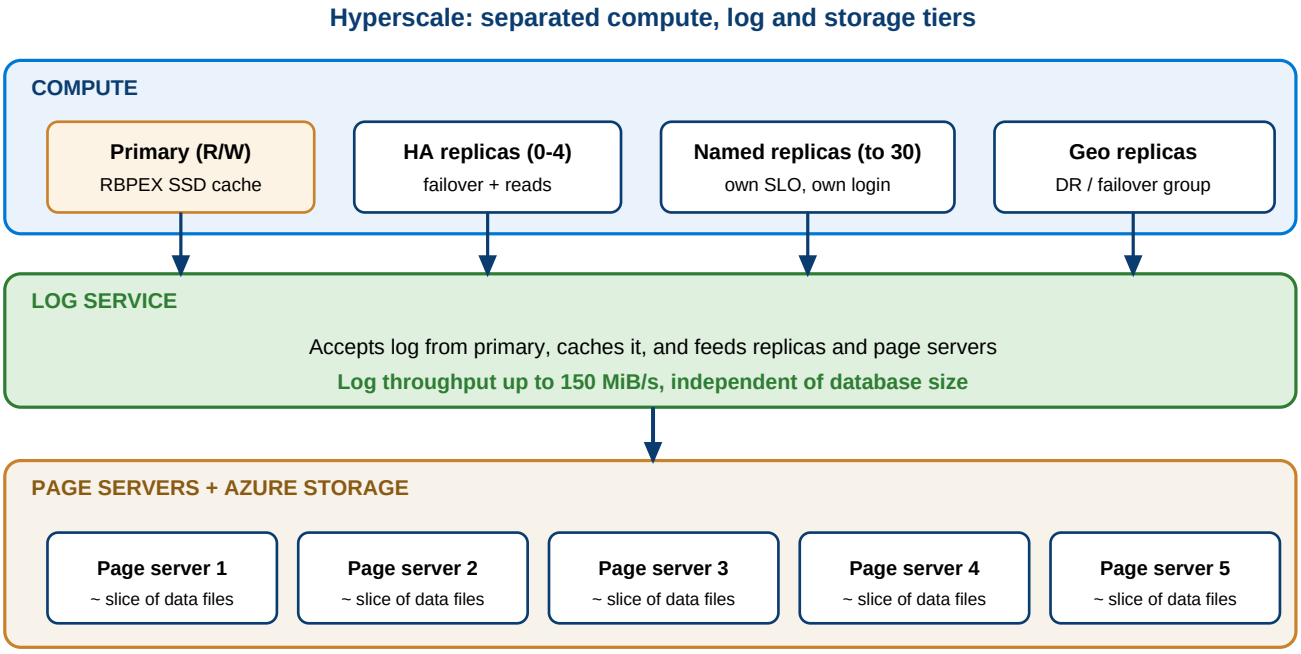
Q12. How is Azure SQL Database billed?

Under the **vCore model** you pay for compute (provisioned per hour or serverless per second), storage (data and log), and backup storage beyond the free allowance equal to the database size. Hyperscale charges for compute, storage used, and replicas. Under **DTU** you pay a bundled price per tier and size. Savings levers include reserved capacity (1 or 3 years), Azure Hybrid Benefit for eligible SQL Server licenses, serverless auto-pause for idle workloads, and elastic pools for many small databases.

Part 2. Service tiers, compute and scaling

Feature	General Purpose	Business Critical	Hyperscale
Storage design	Remote storage, stateless compute	Local SSD, 4-node replica set	Page servers + log service + RBPEX cache
Max size	About 4 TB	About 4 TB	Up to 128 TB
Read replicas	None built in (use geo replica)	1 readable secondary included	Up to 4 HA replicas and up to 30 named replicas
Log throughput	Governed by size and vCores	Governed by size and vCores	Up to 150 MiB/s, independent of size
Backups and restore	Full, diff, log; restore time grows with size	Full, diff, log; restore time grows with size	Snapshot based; fast restore regardless of size
Serverless	Yes, with auto-pause	No	Yes, no auto-pause
Typical use	Most business workloads	Latency-sensitive OLTP, in-memory OLTP	Large databases, high write throughput, scale-out reads

Hyperscale architecture



Tiers, purchasing models and scaling

Q13. DTU vs vCore: which do you recommend and why?

vCore for almost every production design because it separates compute and storage, shows hardware generation, supports Hybrid Benefit and reservations, serverless, Hyperscale and zone redundancy, and gives transparent limits. DTU is a bundled blended measure (CPU, memory, IO) that is simple for small or legacy workloads but hard to map to hardware and not available for Hyperscale. I migrate DTU databases to vCore when I need more control or cost levers.

Q14. General Purpose, Business Critical or Hyperscale: how do you choose?

I decide from workload characteristics: storage latency tolerance, data size and growth, write throughput, read scale-out needs, RTO and cost. General Purpose fits most workloads at lowest cost. Business Critical for very low storage latency, in-memory OLTP or a free readable secondary. Hyperscale when data is beyond a few TB, when log throughput or fast restores matter, or when I need many readers through named replicas. For borderline cases I benchmark with a production-like workload and compare cost per transaction, not just the price list.

Q15. Explain Hyperscale architecture and why it scales.

Compute nodes run the engine and keep a local SSD cache (RBPEX). A **log service** accepts the primary's log and feeds replicas and page servers. **Page servers** own slices of the data files, serve pages to compute and replay the log. Azure Storage holds durable data and snapshots. Because storage is separate, you can scale compute independently, add replicas without copying data, restore from snapshots quickly, and avoid classic log growth issues from slow replicas.

Q16. What are the limitations and considerations for Hyperscale?

Cost model differs (storage and replicas), serverless Hyperscale has no auto-pause, some features and limits vary (for example certain DTU and legacy options are not available), tempdb and workers follow vCore limits, and cold reads after failover can be slower until caches warm. Reverse migration back to General Purpose is supported within a limited window after migration, so I validate current rules before committing. Log rate can still be throttled if a named replica or geo secondary is undersized, so I size replicas deliberately.

Q17. When would you use serverless, and what are the gotchas?

For intermittent, unpredictable or dev/test workloads where you can tolerate a resume delay. You set min and max vCores, and billing is per second of compute used, with optional auto-pause (General Purpose) after an idle delay. Gotchas: first connection after pause can take longer and may fail without retry logic, memory is reclaimed when idle so cache warm-up affects performance, and steady 24x7 workloads are usually cheaper on provisioned compute with reservations. Some features keep the database active and prevent pause.

Q18. When do elastic pools make sense, and how do you size them?

When many databases have low average usage and non-overlapping peaks, such as database-per-tenant SaaS. You size using aggregate peak and average utilization, then set per-database min and max limits to prevent noisy neighbors. I review pool metrics regularly, move busy tenants out to their own compute, and keep placement rules by tier or region. Pools simplify cost but need governance of per-database caps.

Q19. How does scaling up and down work, and what is the impact?

Scaling changes service objective by provisioning new compute and switching over, with a brief connection drop at the end. In General Purpose and Business Critical, time depends on size because data may need to be copied; in Hyperscale scaling compute is much faster because storage is separate. I schedule scale operations outside peak, ensure retry logic, and automate with Azure Automation, Logic Apps or Functions based on metrics or calendar patterns.

Q20. How do you scale reads?

Business Critical includes one readable secondary (use `ApplicationIntent=ReadOnly`). Hyperscale offers HA replicas and up to 30 named replicas that can have different compute sizes and even separate logins. Geo-replicas can serve reads in another region. I route reporting and read-heavy workloads using connection strings and confirm that read consistency requirements (slight lag) are acceptable.

Q21. What resource limits do you worry about beyond CPU and memory?

Worker threads and sessions (errors 10928 and 10929), log rate governance (wait type LOG_RATE_GOVERNOR), data and log IOPS and throughput, tempdb size and tempdb log, max storage (error 40544), concurrent logins, and for Hyperscale replica catch-up throttling (HADR_THROTTLE_LOG_RATE waits). I watch Azure Monitor metrics such as `workers_percent`, `log_write_percent` and `data_io_percent` alongside CPU.

Q22. What hardware options exist and when do they matter?

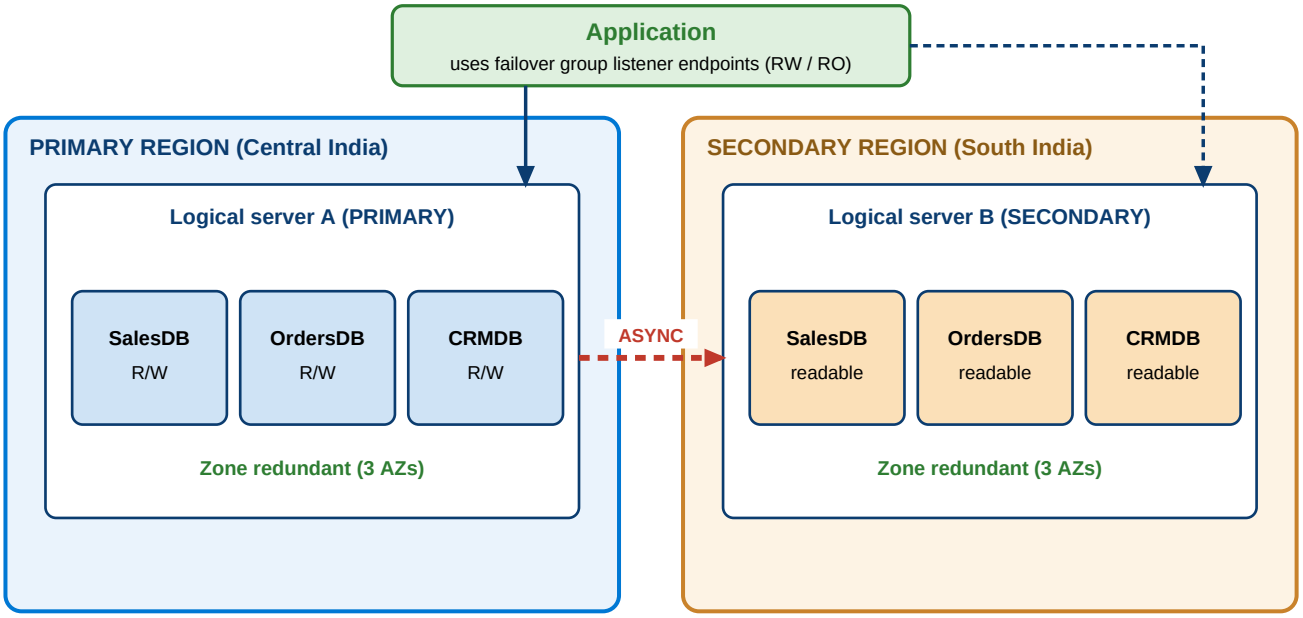
Standard-series (Gen5) is the default; premium-series and memory-optimized premium-series give more memory per vCore and newer CPUs, helpful for memory-hungry or CPU-sensitive workloads; DC-series provides confidential computing for secure enclaves. Availability varies by region and tier, so I verify before designing and revisit when new generations arrive because they often improve price-performance.

Q23. How do you size a database when migrating to Azure SQL Database?

Collect representative performance data (CPU, IO, memory, log rate) over a full business cycle, run an assessment tool that gives SKU recommendations, then map to a tier and test with replay or load tests. I size for p95 or p99 rather than average, leave headroom for growth, and plan to rightsize after 2 to 4 weeks of production telemetry. I include IO latency needs and log throughput, which are often the real constraint.

Part 3. High availability, DR and backups

DR architecture with a failover group



Failover group = one switch for many databases. RPO is seconds (async). Automatic or manual failover.

Business continuity

Q24. Explain RPO, RTO and the SLA options for Azure SQL Database.

RPO is acceptable data loss and RTO is acceptable downtime. Azure SQL Database provides a high SLA within a region, with a higher figure when zone redundancy is enabled on supported tiers. Within a region, HA failover is automatic and data loss is zero for committed data. For regional DR, geo-replication is asynchronous so RPO is typically seconds, not zero, and RTO depends on failover process (automatic policy or manual). I always check the current SLA document and translate it to application-level targets, including dependencies.

Q25. What does zone redundancy protect against and what does it cost?

It spreads replicas across availability zones in one region so that a datacenter (zone) failure does not take the database down, without data loss and with automatic failover. It does not protect against a full region outage, human error or logical corruption. It increases cost for some tiers and is not available in every region or SKU, so I enable it for tier-1 workloads and pair it with geo-replication for regional DR.

Q26. Compare PITR, LTR, geo-restore, active geo-replication and failover groups.

PITR restores a database to any moment in the retention window (default 7 days, configurable 1 to 35) as a new database. LTR keeps weekly, monthly or yearly full backups up to 10 years for compliance. Geo-restore uses geo-redundant backups to recreate a database in another region with higher RPO and RTO. Active geo-replication creates up to four readable secondaries with async replication, with failover per database. Failover groups manage a set of databases together and provide stable read-write and read-only listener endpoints. I combine them based on RPO, RTO and compliance requirements.

Q27. Failover groups vs active geo-replication: which and when?

Failover groups for most application designs: they fail over many databases together, expose listener endpoints so connection strings do not change, and support automatic failover policy with a grace period. Active geo-replication when I need per-database control, multiple secondaries or targeted read scale in several regions. Failover groups apply at the logical server level and are the recommended approach for enterprise DR.

Q28. Design DR for a tier-1 application with RPO under 5 seconds and RTO under 1 minute.

Business Critical or Hyperscale with zone redundancy in the primary region, a failover group to a paired or chosen secondary region with the same SLO, private networking and DNS configured in both regions, application tier deployed active-passive or active-active, and automated runbooks for failover. I use async replication's typical seconds-level RPO, monitor replication lag, test failover quarterly, and set an automatic policy only if the business accepts potential data loss. The real RTO includes app, DNS and dependencies, so I measure end to end.

Q29. How do automatic backups work and what changes for Hyperscale?

Full backups roughly weekly, differentials roughly every 12 to 24 hours, and log backups about every 10 minutes enable PITR within the retention period. Backup storage redundancy can be local, zone, geo or geo-zone redundant. Hyperscale uses storage snapshots and log retention, so backups are near-instant and restores are fast regardless of size. I choose redundancy to meet data residency and DR needs, and watch backup storage cost for large or high-churn databases.

Q30. A developer truncated a production table at 2 PM. How do you recover?

Stop further damage, identify the exact time of the mistake from logs, audit or application records, restore the database via PITR to a new database at a time just before the event, validate the data, and then either swap names or copy only the affected table back. For minimal disruption I prefer extracting the table from the restored copy and merging it, rather than replacing the live database. Afterwards I add guardrails: least privilege, change control, and a ledger or audit if appropriate.

Q31. How do you restore a dropped database or restore across regions?

A dropped database can be restored from the deleted-databases list within the retention window. Cross-server and cross-subscription restores are possible via copy or restore operations. For region loss, geo-restore from geo-redundant backups or fail over a geo-replica or failover group. I document the exact steps and test them because permissions and networking often cause surprises.

Q32. How do you handle long-term retention and compliance?

Define weekly, monthly and yearly retention policies aligned to regulation, store LTR backups in a redundancy level that meets residency rules, and periodically restore test them. Combine with immutability or legal-hold policies where applicable and keep evidence of restore tests for auditors.

Q33. What does a proper DR drill include?

Pre-checks (replication lag, health, runbook, access), a planned failover (no data loss) and a forced failover scenario, application and DNS validation, post-failover tasks (logins, firewall and private endpoints, jobs, secrets, monitoring), failback, and a report with measured RTO and RPO and action items. Drills should be scheduled, repeated, and run by the people who will respond in a real incident.

Q34. What networking issues commonly break DR for failover groups?

Private endpoints and DNS zones configured only in the primary region, firewall or VNet rules missing on the secondary server, different outbound connectivity or NSG rules, and application configuration pointing to server names rather than listener endpoints. I standardize infrastructure as code for both regions and test connectivity from the DR application tier.

Part 4. Security and compliance

Security design

Q35. Describe your defense-in-depth strategy for Azure SQL Database.

Network: private endpoints, deny public access, minimal TLS version. Identity: Entra ID only authentication, managed identities, MFA and Conditional Access, least privilege roles. Data protection: TDE (optionally customer-managed key), Always Encrypted for sensitive columns, TLS in transit. Data access controls: row-level security, dynamic data masking, column permissions. Monitoring: auditing to Log Analytics, Defender for SQL, alerting. Governance: Azure Policy, tagging, regular access reviews. I treat each layer as independent so a single misconfiguration does not expose data.

Q36. What is your authentication recommendation?

Use Microsoft Entra authentication, ideally **Entra-only** mode to disable SQL authentication, with application access through managed identities rather than passwords. Admins use groups and PIM for just-in-time elevation. Where SQL authentication is unavoidable, store secrets in Key Vault, rotate them, and restrict by network.

Q37. Private Link vs service endpoints vs firewall rules?

Private Link (private endpoint) gives a private IP for the server inside your VNet, works from on-prem over VPN or ExpressRoute, and lets you disable public access, which makes it the preferred option. Service endpoints extend VNet identity to the service but traffic still goes to the public endpoint. Firewall rules are IP-based allow lists and are weakest. I combine private endpoint with DNS design and deny public network access.

Q38. Explain TDE, customer-managed keys and Always Encrypted. How are they different?

TDE encrypts data and log files and backups at rest, transparent to the application, with service-managed keys by default. CMK (BYOK) lets you hold the TDE protector in Key Vault for control, rotation and revocation. Always Encrypted protects specific columns end to end: keys are held by the client, so even DBAs and Microsoft cannot see plaintext. Secure enclaves extend what you can do on encrypted data (such as range and pattern queries) inside a protected enclave.

Q39. What happens if a CMK is revoked or deleted?

The database becomes inaccessible because it cannot decrypt the key hierarchy, which is the intended behavior for revocation. Recovery requires restoring key access in Key Vault (soft delete and purge protection should be enabled). I design key rotation, access policies or RBAC, and monitoring on the Key Vault, and I test recovery before relying on CMK.

Q40. Row-level security, dynamic data masking or column permissions: when do you use each?

RLS enforces which rows a user can see, ideal for multi-tenant shared tables. Column permissions or views restrict who can query sensitive columns. DDM obfuscates results for non-privileged users but is not a security boundary because users can infer values, so it complements, not replaces, permissions or encryption.

Q41. How do you implement auditing and threat protection?

Server-level auditing to Log Analytics (and storage for retention), Defender for SQL for vulnerability assessment and anomaly detection, alerts into a SIEM such as Sentinel, and regular review of audit logs for privileged actions. I define retention by regulation and avoid logging noise by tuning audit action groups.

Q42. What is SQL Ledger and when is it useful?

Ledger provides tamper-evidence through cryptographic hashes in blockchain-like structures, with updatable and append-only ledger tables and a verification process using digests stored externally. It is useful for audit trails, financial records and multi-party trust, and complements, but does not replace, access control.

Q43. How do you secure a multi-tenant SaaS database?

Choose an isolation model (database per tenant, shared schema with TenantId, or hybrid). For shared schema I use RLS with session context, strict app roles, and per-tenant keys where required. For database per tenant I use pools and automate provisioning. Consider noisy neighbor, backup and restore per tenant, data residency and offboarding. I always test tenant isolation with negative tests.

Q44. How do you handle compliance and data residency?

Select regions and backup redundancy to keep data in required geographies, review geo-replication destinations, apply Azure Policy to enforce settings (public access disabled, TDE, auditing), classify data with sensitivity labels, and keep audit evidence. I map controls to the frameworks the company follows (ISO 27001, SOC 2, PCI DSS, HIPAA, GDPR) and keep documentation current.

Q45. How do you separate duties and manage privileged access?

Use Entra groups for roles, PIM for time-bound admin access, no shared accounts, break-glass accounts monitored by alerts, database roles like db_datareader and custom roles instead of db_owner, and review access regularly. Developers get access in non-prod and read-only or just-in-time access in prod.

Part 5. Performance tuning and troubleshooting

Performance methodology

Q46. A critical application is slow. What is your step-by-step approach in Azure SQL Database?

1) Define scope and timing: what is slow, since when, what changed (deployment, data growth, tier change, maintenance). 2) Check platform health and resource metrics (CPU, workers, IO, log rate) in Azure Monitor and `sys.dm_db_resource_stats`. 3) Use Query Store to find top consumers and regressions. 4) Review waits and blocking. 5) Check for throttling signals (log rate governance, resource limits, tempdb). 6) Apply a targeted fix (plan force, index, query change, scale), measure, and document. 7) Add alerts and a runbook so the next occurrence is faster.

Q47. How do you use Query Store and automatic tuning effectively?

Query Store is on by default and tracks runtime stats and plans. I use it to find regressions, force a known good plan as a stop-gap, and compare before and after. Automatic plan correction detects regression and forces the last good plan. Automatic index management can create or drop indexes, but I review its recommendations in critical systems before enabling auto-apply. I set cleanup and capture mode to control storage.

Q48. Which DMVs and metrics do you rely on, and how do sys.dm_db_resource_stats and sys.resource_stats differ?

`sys.dm_db_resource_stats` (in the user database) has about 15-second granularity for roughly the last hour, good for live incidents. `sys.resource_stats` (in master) has about 5-minute granularity for about 14 days, good for trend and sizing. For waits I use `sys.dm_db_wait_stats` (database scope), and for requests `sys.dm_exec_requests`, `sys.dm_exec_query_stats` and Query Store views.

Q49. Walk through diagnosing sustained high CPU.

Confirm via metrics that CPU is near the limit and see whether it is steady or spiky. In Query Store sort by CPU and look for new or regressed queries, plan changes and parameter sniffing. Check missing indexes, scans, implicit conversions and excessive recompiles. Look at concurrency (many executions of a cheap query). Mitigate by plan forcing, index or query changes, caching, and if needed scale up temporarily. Verify afterward that CPU dropped for the right reasons.

Q50. What is log rate governance and how do you fix related issues?

Azure SQL Database throttles log generation to stay within the tier's limit and to keep replicas in sync, shown as LOG_RATE_GOVERNOR or HADR_THROTTLE_LOG_RATE waits and high `log_write_percent`. Fixes: batch and tune large writes, avoid unnecessary index rebuilds, use minimal logging patterns where possible, scale to a tier with higher log rate (Hyperscale offers up to 150 MiB/s), ensure named or geo replicas are not undersized and slowing the primary, and schedule bulk loads off-peak.

Q51. Errors 10928 and 10929 appear. What do they mean and what do you do?

The database reached its limit for concurrent workers or sessions (10928) or the request limit (10929) for its service objective. I find which queries are blocked or long-running and holding workers, fix blocking and inefficient queries, add connection pooling limits in the application, and scale up if demand is legitimate. I also alert on `workers_percent` before it reaches 100.

Q52. How do you handle blocking and deadlocks?

RCSI is on by default, which reduces reader-writer blocking, but writer-writer blocking and long transactions still occur. I identify head blockers with DMVs, shorten transactions, add proper indexes, and fix access order for deadlocks. Deadlock graphs can be captured through Extended Events or seen in the database-level diagnostics. I also review application retry behavior for deadlock victims (error 1205).

Q53. What tempdb considerations exist in Azure SQL Database?

tempdb size and log limits depend on the service objective and cannot be configured directly. Spills, large sorts, version store from long transactions under RCSI, and temp table abuse can exhaust it. I monitor tempdb metrics, fix spills with better indexing and memory grants, avoid long-running transactions, and scale up if the workload requires more tempdb capacity.

Q54. How do you deal with parameter sniffing?

Identify via Query Store (same query, multiple plans with different durations). Options: parameter sensitive plan optimization on supported compatibility levels, OPTION (RECOMPILE) for volatile cases, OPTIMIZE FOR hints, query rewrite, better statistics or indexing, plan forcing as a temporary fix, and splitting procedures by scenario. I prefer fixes that are stable and documented over long-term forced plans.

Q55. What is your indexing strategy in the cloud?

Design indexes from workload data, not guesses: use Query Store, missing index DMVs as hints only, and test on a copy. Maintain a balance of read benefit vs write cost and storage cost. Use filtered indexes and columnstore where appropriate. Use online and resumable index operations in production, and watch log rate impact for rebuilds. I review automatic index recommendations regularly rather than trusting them blindly.

Q56. What engine features are worth knowing: IQP, in-memory OLTP, columnstore?

Intelligent Query Processing (adaptive joins, memory grant feedback, batch mode on rowstore, PSP optimization) improves performance with compatibility level upgrades. Columnstore is available for analytics and HTAP. In-memory OLTP is available in premium or Business Critical tiers and suits very high-throughput OLTP with specific patterns. I confirm availability per tier.

Q57. List the transient errors you handle in code and the retry approach.

Examples: 40613 (database not currently available), 40501 (service busy), 40197 (service error), 49918 and related resource errors, 10928 and 10929 (limits, retry carefully), and network or login timeouts. Use a retry policy with exponential backoff and jitter, limited attempts, circuit breaker for persistent failure, and idempotent operations. Libraries like EF Core's SqlServerRetryingExecutionStrategy or Polly help.

Q58. How do you size or baseline a workload with spiky and unpredictable usage?

Capture at least a full business cycle of metrics, analyze p95 and p99 and burst duration, consider serverless with proper min and max or an elastic pool, and decide between scaling on schedule or on metrics. Avoid sizing for the rare absolute peak without considering caching or queueing; tune the worst queries first.

Q59. How do you design monitoring and alerting?

Metrics alerts on CPU, workers, log write, data IO, storage, deadlocks, failed connections and replication lag; log-based alerts from audit and diagnostics in Log Analytics; dashboards for each application; and clear on-call runbooks. Alerts are tuned to avoid fatigue, and each alert has an owner and a documented action.

Part 6. Migration and modernization

Migration strategy

Q60. Describe your end-to-end approach to migrating on-prem SQL Server to Azure SQL Database.

Discover and inventory, assess compatibility and performance, choose target (SQL Database, MI or VM) per database, design landing zone (network, identity, security, monitoring), choose migration method, run dress rehearsals, plan cutover and rollback, execute with validation, then optimize and decommission. I include stakeholders for downtime windows, dependencies (jobs, linked servers, SSIS, reports) and cost forecasts.

Q61. Which tools and methods do you use, and how do you choose?

Azure Migrate and the Azure Data Studio migration extension (formerly DMA) for assessment and SKU recommendations. Azure Database Migration Service for offline migrations of schema and data. SqlPackage (DACPAC and BACPAC) for smaller databases. Transactional replication with SQL Database as subscriber, or third-party tools, for low-downtime scenarios. Azure Data Factory or bulk copy for data movement and transformation. I choose based on size, allowed downtime, feature compatibility and team skills.

Q62. How do you achieve minimal downtime to Azure SQL Database?

SQL Database does not support native .bak restore or the MI link. Options: transactional replication into SQL Database, DMS or ADF with incremental loading and a short cutover freeze, or application-level dual writes in advanced cases. If true near-zero downtime is mandatory for a database that needs instance features, Managed Instance with Link or Log Replay Service may be the better target.

Q63. How do you handle features that SQL Database does not support (Agent jobs, cross-database queries, CLR, linked servers)?

Replace Agent jobs with Elastic Jobs, Azure Automation, Data Factory or Logic Apps. Refactor cross-database dependencies by consolidating schemas into one database or using elastic query for light needs. Rewrite CLR logic into application or T-SQL where possible. Replace linked servers with application integration, external tables or ETL. If refactoring cost is too high, I recommend Managed Instance.

Q64. You must migrate 500 databases. How do you plan it?

Group by complexity, size, dependencies and business criticality into waves. Automate assessment and provisioning with scripts and infrastructure as code, standardize security and monitoring, use pools or tiers based on usage data, and build a repeatable migration pipeline with validation tests. Pilot with a few databases, measure effort and defects, then scale. Maintain a dashboard of status and a rollback plan per wave.

Q65. How do you validate a migration and ensure performance parity?

Compare row counts and checksums, run application smoke and regression tests, replay workload or run benchmarks, compare Query Store baselines, check security settings and jobs, and monitor closely for a few weeks. I keep the source in read-only mode for a defined fallback window and define success criteria before cutover.

Q66. What are the considerations for migrating to Hyperscale?

Validate size and growth, test log rate and workload on Hyperscale, plan for named replicas and cost model, confirm feature compatibility, and understand reverse migration limits. Migration can be done by changing the service tier from another SQL Database tier (data size operation) or by standard migration tools. I run a proof of concept comparing cost and performance with Business Critical.

Q67. When would you advise against Azure SQL Database?

When the application depends on instance-level features that are costly to refactor, requires OS access, needs cross-database transactions across many databases, has very tight near-zero downtime migration needs with .bak-based processes, or the vendor does not support PaaS. In these cases MI or SQL on VM is the better fit, and I document the reasoning.

Q68. How do you integrate SQL Database with analytics and downstream systems?

Options include Change Tracking or CDC feeding ADF or event pipelines, Mirroring of Azure SQL Database into Microsoft Fabric for near-real-time analytics, readable replicas for reporting, and Hyperscale named replicas for heavy read workloads. I avoid running heavy analytics on the primary and choose a pattern based on latency and cost.

Part 7. Operations, DevOps, governance and cost

Operations and platform engineering

Q69. How do you manage schema changes and CI/CD for Azure SQL Database?

Keep the schema in source control as a SQL project (DACPAC) or migration scripts, build in a pipeline, run unit and deployment tests on an ephemeral database, and deploy via SqlPackage or the Azure SQL deploy action with approvals. I use drift detection, backward-compatible changes (expand and contract), feature flags for risky changes, and a rollback plan. Pre-deployment checks include blocking data-loss operations.

Q70. Which Infrastructure as Code approach do you use?

Bicep or Terraform for servers, databases, pools, failover groups, private endpoints, auditing and policies, with modules for standards. Parameter files per environment, pull request reviews, and policy checks in the pipeline. This gives repeatable DR environments and avoids drift.

Q71. What replaces SQL Server Agent in Azure SQL Database?

Elastic Jobs for T-SQL jobs across many databases, Azure Automation or Functions for scripted tasks, Data Factory for orchestration, and Logic Apps for event-driven flows. Maintenance tasks like index and statistics maintenance are still useful but run through these tools or smart policies; backups and integrity checks are handled by the platform.

Q72. How do you govern Azure SQL at scale?

A landing zone with management groups, Azure Policy to enforce baselines (no public access, Entra-only, auditing, TDE, tags, allowed regions and SKUs), RBAC and PIM, central logging, cost management budgets and alerts, and a service catalog with approved patterns (standard pools, tiers, DR options). Exceptions follow a documented approval process.

Q73. Share your cost optimization playbook for Azure SQL Database.

1) Rightsize using telemetry (CPU, IO, log) and Query Store. 2) Use reservations for steady workloads and Azure Hybrid Benefit. 3) Use serverless or pause for dev/test, schedule scale down. 4) Consolidate many small databases into elastic pools. 5) Tune the most expensive queries before scaling. 6) Review backup retention and redundancy, and LTR policies. 7) Evaluate Hyperscale or General Purpose per workload rather than defaulting to Business Critical. 8) Delete orphaned databases and replicas. I report savings with before and after numbers.

Q74. How do you handle capacity planning and automation of scaling?

Track growth in data size, log rate and CPU by trend, forecast quarterly, and set thresholds for review. Automate scale actions through Automation, Functions or Logic Apps tied to metric alerts or schedules, with guardrails on maximum SKU and cost. Always test that apps tolerate the brief disconnect at scale time.

Q75. How do you handle planned maintenance and change management?

Define preferred maintenance windows, publish change calendars, notify application owners, use Resource Health and Service Health alerts, and keep runbooks. For my own changes, I use standard, normal and emergency categories with approvals, test in staging and ensure rollback.

Q76. Describe patterns for multi-tenant data architecture on Azure SQL.

Database-per-tenant in elastic pools gives strong isolation and per-tenant restore but increases object count and management; shared database with TenantId and RLS is cheapest but risks noisy neighbors and complex restores; sharded designs use the Elastic Database client library and shard maps; hybrid approaches put large tenants on dedicated databases. I decide based on tenant count, size distribution, isolation, compliance and operational maturity.

Q77. What is the status of Azure SQL Data Sync, and what do you use instead?

Azure SQL Data Sync has been announced for retirement (September 2027), so I verify the current notice and avoid new designs on it. Alternatives include transactional replication from supported sources, Change Tracking or CDC with ADF, Fabric Mirroring or event-driven pipelines, and application-level sync depending on the scenario.

Part 8. Architecture and scenario-based questions

Use this structure for scenarios: **clarify requirements** (RPO, RTO, size, growth, compliance, budget), **propose architecture**, **explain trade-offs**, **cover security, operations and cost**, and **state risks and validation steps**.

Scenarios

Q78. Design a multi-tenant SaaS platform with 2,000 tenants of widely varying sizes.

Start with a hybrid model: database-per-tenant in elastic pools for the long tail, dedicated databases (or Hyperscale) for large tenants, a catalog database for tenant metadata and routing, automated provisioning via IaC, per-tenant restore and export, and per-pool resource caps to avoid noisy neighbors. Add monitoring per tenant, a tenant-move process between pools, and tiered pricing aligned to cost. Security uses Entra-based service identities, private endpoints and auditing.

Q79. A global application needs low-latency reads in three regions and a safe write path.

Single write region with failover group for DR, read replicas via geo-replication or Hyperscale named replicas in each read region, application routing of read traffic to the nearest replica, and clear handling of read lag. If writes must be local in each region, discuss a data partitioning strategy or a different data architecture, because Azure SQL Database is single-writer per database. Be explicit about consistency trade-offs.

Q80. A bank needs RPO near zero, strict compliance and data residency. What is your design?

Business Critical with zone redundancy, customer-managed keys, private endpoints only, Entra-only auth with PIM, auditing to immutable storage, ledger tables for critical records, geo-replication within the allowed jurisdiction, and LTR for regulatory retention. Be honest that cross-region replication is async so RPO is not strictly zero across regions; if zero is mandatory, the design needs synchronous mechanisms within a region or application-level safeguards. Include DR drills and evidence.

Q81. Performance regressed right after a release. What do you do?

Compare Query Store before and after the release window, identify regressed queries and plan changes, check for new parameter patterns, missing indexes or schema changes. Force the previous plan as a quick mitigation if safe, then fix the code or index and remove forced plans afterward. If necessary, roll back the release. Add a performance gate in CI/CD using Query Store comparisons.

Q82. The monthly bill doubled. How do you investigate and fix it?

Use Cost Management to break down by resource, tier, storage, backup and egress. Look for scale-ups left in place, new databases or replicas, serverless databases that no longer pause, backup storage growth, and Hyperscale storage growth. Right-size using metrics, apply reservations and Hybrid Benefit, delete orphaned resources, tune heavy queries, and add budgets and alerts with owners.

Q83. Design a 50 TB OLTP and reporting platform on Azure SQL Database.

Hyperscale with enough vCores for OLTP, HA replicas for failover and read scale, named replicas for reporting isolated from the primary, partitioning and archival strategy for historical data, columnstore for analytical tables, careful log rate planning, zone redundancy, a failover group for DR, and Fabric Mirroring or ADF for downstream analytics. Validate with load tests, and monitor log rate and replica catch-up.

Q84. Data was corrupted by a faulty batch job overnight and discovered 6 hours later. What is the recovery plan?

Freeze the job, PITR to a new database at the last good time, compare and reconcile data (differences for affected tables), repair in place with scripts where only part of the data was affected, validate with business owners, and document. Use audit logs to define the exact timeline. Add safeguards: transaction control, dry-run mode, and backup/restore tests as part of the release process.

Q85. How would you design dev, test and prod environments cost-effectively?

Dev and test on serverless or small provisioned SKUs with auto-pause where possible, shared elastic pools, automated environment creation from DACPAC or PITR copies with masked data, schedule scale-downs, and production-like staging only when needed for performance tests. Production uses reservations and DR. Policy enforces tags and lifetimes for ephemeral databases.

Q86. Plan a weekend cutover for a 2 TB database to Azure SQL Database.

Pre-stage with a full load and incremental sync, rehearse the cutover at least twice, define a freeze time, final sync, validation checks, DNS or connection string switch, smoke tests, and a go/no-go checkpoint. Rollback: keep the source read-only and ready, with a clear trigger. Communicate with stakeholders, have the app, network, security and DB teams on a bridge, and monitor intensively after go-live.

Q87. After a failover group failover, some users still hit the old primary. Why?

Applications using the primary server name instead of the failover group listener, cached DNS with long TTL, connection pools holding old connections, or private DNS not updated in the secondary region. Fix by using the listener endpoints, setting appropriate pool lifetimes, validating DNS and private endpoint configuration, and adding a post-failover validation script.

Part 9. Leadership and behavioral questions

Use the **STAR** format: Situation, Task, Action, Result. At 11+ years the result should include numbers and what you changed permanently.

Behavioral and leadership

Q88. Tell me about the most serious production incident you handled.

Choose an incident with real impact. Structure: what failed and the business impact, how you triaged (metrics, logs, Query Store, change history), who you coordinated (app, network, vendor, management), the fix and time to recovery, and the lasting improvements (alerts, runbook, automation, design change). Quote duration and customer impact and show calm, structured leadership.

Q89. How do you set standards and mentor your team?

Create reference architectures, coding and indexing guidelines, review checklists, and templates for IaC. Run knowledge sessions and pair on incidents, give juniors safe ownership of small improvements, and track team metrics such as incident repeat rate and review turnaround. Show an example where a mentee grew into a larger role.

Q90. Describe a time you convinced stakeholders to invest in resilience or security.

Frame it in business terms: risk, cost of downtime, compliance exposure, and options with cost. Use data from a DR drill or incident, propose phased delivery, and show results such as improved RTO or audit findings closed.

Q91. Tell me about a cost optimization you delivered.

Give a concrete example: baseline spend, analysis performed, actions (rightsizing, reservations, pooling, query tuning, retention changes), risks managed, and the verified savings percentage with no impact on SLOs. Mention how you embedded cost review into a recurring process.

Q92. How do you work with application developers who resist DBA controls?

Build trust through enablement: self-service pipelines with guardrails, fast feedback on queries, shared dashboards, and clear reasons for rules. Escalate only for real risks and provide alternatives. Show an example where collaboration improved performance or delivery speed.

Q93. How do you stay current with Azure SQL changes?

Follow Azure Updates and the Azure SQL blog, release notes, Microsoft Learn, community experts, and run small proofs of concept for relevant features. Maintain an internal tech radar and share summaries with the team so that decisions are reviewed when features reach GA or are retired.

Q94. How do you decide between PaaS and IaaS for a database workload?

Evaluate requirements (features, control, compliance), operational capacity, cost at scale, risk, and strategic direction. Prefer PaaS to reduce undifferentiated operations unless a clear constraint exists, and document exceptions with an exit path.

Q95. What would be your first 90 days in this role?

Days 1 to 30: learn the estate, stakeholders, incidents, costs and risks; quick wins like alerts and backup validation. Days 31 to 60: baseline and prioritize improvements (security posture, DR gaps, performance hotspots, cost); define standards. Days 61 to 90: deliver two or three visible improvements, set up a roadmap, KPIs, and a regular operating cadence with the teams.

Part 10. Cheat sheet

Platform comparison

Area	Azure SQL Database	Managed Instance	SQL Server on VM
Scope	Single database or pool	Instance with many databases	Full SQL Server instance

Area	Azure SQL Database	Managed Instance	SQL Server on VM
Compatibility	High, some instance features missing	Near 100% instance features	100%
Network	Public endpoint or Private Link	Private IP in your VNet	Your VNet and NIC
Management	Lowest overhead	Low overhead	You manage OS and SQL
Best for	New cloud apps, SaaS, elastic workloads	Lift and shift with minimal change	Max control, special software

Key numbers to remember (verify current values)

Topic	Value
Default PITR retention	7 days (configurable 1 to 35 days)
LTR maximum	Up to 10 years
Hyperscale max size	128 TB
Hyperscale log throughput	Up to 150 MiB/s
Hyperscale replicas	0 to 4 HA replicas, up to 30 named replicas
Active geo-replication	Up to 4 secondaries
Default connection port	1433 (plus redirect port ranges)
Backups cadence	Weekly full, 12 to 24 hour differential, about 10 minute log
Default MAXDOP (single DB)	8

Common error codes

Error	Meaning	Typical action
40613	Database not currently available	Retry with backoff; check failover or maintenance
40501	Service is busy	Retry after delay; check throttling
40197	Service error processing request	Retry; check Resource Health
40544	Database reached its size quota	Increase max size or archive data
40552	Session terminated due to excessive log space use	Break up large transactions
40549	Session terminated due to long-running transaction	Shorten transactions
40615	Client IP not allowed by firewall	Fix firewall or use private endpoint
10928 / 10929	Resource limit reached (workers, sessions, requests)	Fix blocking, tune, scale up
1205	Deadlock victim	Retry transaction, fix access order

Handy queries

```
-- Recent resource usage (about 1 hour, 15-second granularity)
SELECT TOP 20 end_time, avg_cpu_percent, avg_data_io_percent,
       avg_log_write_percent, avg_memory_usage_percent, max_worker_percent
FROM sys.dm_db_resource_stats ORDER BY end_time DESC;

-- Top CPU queries in Query Store (last 24 hours)
SELECT TOP 10 q.query_id, SUBSTRING(qt.query_sql_text,1,120) AS txt,
       SUM(rs.count_executions) AS execs,
       SUM(rs.avg_cpu_time * rs.count_executions)/1000.0 AS total_cpu_ms
FROM sys.query_store_query q
JOIN sys.query_store_query_text qt ON q.query_text_id = qt.query_text_id
JOIN sys.query_store_plan p ON q.query_id = p.query_id
JOIN sys.query_store_runtime_stats rs ON p.plan_id = rs.plan_id
JOIN sys.query_store_runtime_stats_interval i ON rs.runtime_stats_interval_id = i.runtime_stats_interval_id
WHERE i.start_time > DATEADD(HOUR,-24,SYSUTCDATETIME())
GROUP BY q.query_id, qt.query_sql_text ORDER BY total_cpu_ms DESC;

-- Database-scoped wait stats
SELECT TOP 10 wait_type, wait_time_ms, waiting_tasks_count
FROM sys.dm_db_wait_stats ORDER BY wait_time_ms DESC;

-- Current service objective
SELECT DATABASEPROPERTYEX(DB_NAME(),'Edition') AS edition,
       DATABASEPROPERTYEX(DB_NAME(),'ServiceObjective') AS slo,
       DATABASEPROPERTYEX(DB_NAME(),'MaxSizeInBytes')/1073741824 AS max_gb;
```

Create an Entra user and a managed identity user

```
-- Connect as the Entra admin to the user database
CREATE USER [dba.team@contoso.com] FROM EXTERNAL PROVIDER;
ALTER ROLE db_datareader ADD MEMBER [dba.team@contoso.com];

CREATE USER [my-app-managed-identity] FROM EXTERNAL PROVIDER;
ALTER ROLE db_datareader ADD MEMBER [my-app-managed-identity];
ALTER ROLE db_datawriter ADD MEMBER [my-app-managed-identity];
```

Failover group with Azure CLI (example)

```
az sql failover-group create --name fog-sales \
  --resource-group rg-sql --server sql-primary \
  --partner-server sql-secondary --partner-resource-group rg-sql-dr \
  --add-db SalesDB OrdersDB \
  --failover-policy Automatic --grace-period 1

# Planned failover (run against the secondary server)
az sql failover-group set-primary --name fog-sales \
  --resource-group rg-sql-dr --server sql-secondary
```

Questions to ask the interviewer

- What are the current RPO and RTO targets and when was the last DR drill?
- What does the estate look like: number of databases, tiers, pools, Hyperscale usage, and biggest risks?
- How are security baselines, cost governance and change management handled?
- How is the data platform team organized, and what is the on-call model?
- What are the top three performance or reliability pain points you want the next hire to fix?

30-second pitch template

"I have 11+ years in SQL Server and Azure data platforms. I have designed and operated Azure SQL Database estates, including HA and DR with failover groups, security with Entra-only access and private endpoints, performance tuning with Query Store, and migrations of [N] databases with [X] downtime. In my

last role I [specific result: reduced cost by X percent, improved RTO to Y minutes, cut incidents by Z percent]. I enjoy mentoring teams and setting standards so the platform stays reliable and cost-efficient."